Artificial Neural Network

Instructor: Jyotishka Datta

Virginia Tech

April 19, 2022

The following materials are mostly taken from Taylor B. Arnold's Stat 665 Lecture notes, with some minor modifications to suit our class.

The second edition of ISLR book has a nice chapter on 'deep learning'!

Other resources:

1. Michael A. Nielsen, "Neural Networks and Deep Learning", Determination Press, 2015. [html tutorial]. The backpropagation is described here: http://neuralnetworksanddeeplearning.com/chap2.html.
2. Geoffrey E. Hinton, Yann LeCun, and Yoshua Bengio (video; NIPS 2015): http://research.microsoft.com/apps/video/default.aspx?id=259574

A Simple Decision

Say you want to decide whether you are going to attend a cheese festival this upcoming weekend. There are three variables that go into your decision:

1. Is the weather good?
2. Does your friend want to go with you?
3. Is it near public transportation?

We'll assume that answers to these questions are the only factors that go into your decision.

A Simple Decision, Contd.

I will write the answers to these question as binary variables x_i, with zero being the answer 'no' and one being the answer 'yes':

1. Is the weather good? x_1
2. Does your friend want to go with you? x_2
3. Is it near public transportation? x_3

Now, what is an easy way to describe the decision statement resulting from these inputs.

A Simple Decision, Contd.

We could determine weights ω_i indicating how important each feature is to whether you would like to attend. We can then see if:

$$x_1\omega_1 + x_2\ \omega_2 + x_3\ \omega_3 \geq \text{threshold}.$$

For some pre-determined threshold. If this statement is true, we would attend the festival, and otherwise we would not.

For example, if we really hated bad weather but care less about going with our friend and public transit, we could pick the weights 6, 2 and 2.

For example, if we really hated bad weather but care less about going with our friend and public transit, we could pick the weights 6, 2 and 2.

With a threshold of 5, this causes us to go if and only if the weather is good.

For example, if we really hated bad weather but care less about going with our friend and public transit, we could pick the weights 6, 2 and 2.

With a threshold of 5, this causes us to go if and only if the weather is good.

What happens if the threshold is decreased to 3? What about if it is decreased to 1?

If we define a new binary variable y that represents whether we go to the festival, we can write this variable as:

$$y = \begin{cases} 0, & x_1\omega_1 + x_2\,\omega_2 + x_3\,\omega_3 \leq \text{threshold}, \\ 1, & x_1\omega_1 + x_2\,\omega_2 + x_3\,\omega_3 \geq \text{threshold} \end{cases}$$

Is this starting to look familiar yet?

Now, we can rewrite this in terms of a dot product between the vector of of all binary inputs (x), a vector of weights (ω), and change the threshold to the negative bias (b), we have:

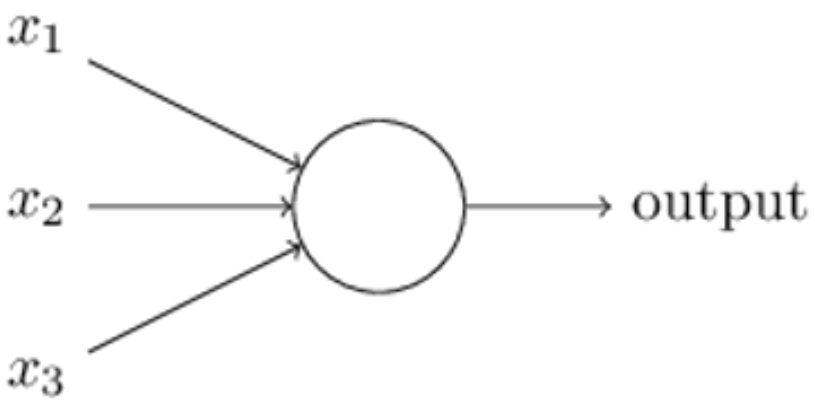
$$y = \begin{cases} 0, & x\omega + b < 0, \\ 1, & x\omega + b \geq 0 \end{cases}$$

So we are really just finding separating hyperplanes again: recall linear decision boundaries we have seen in the context of logistic regression.

This is also seen in support vector machines !

A perceptron

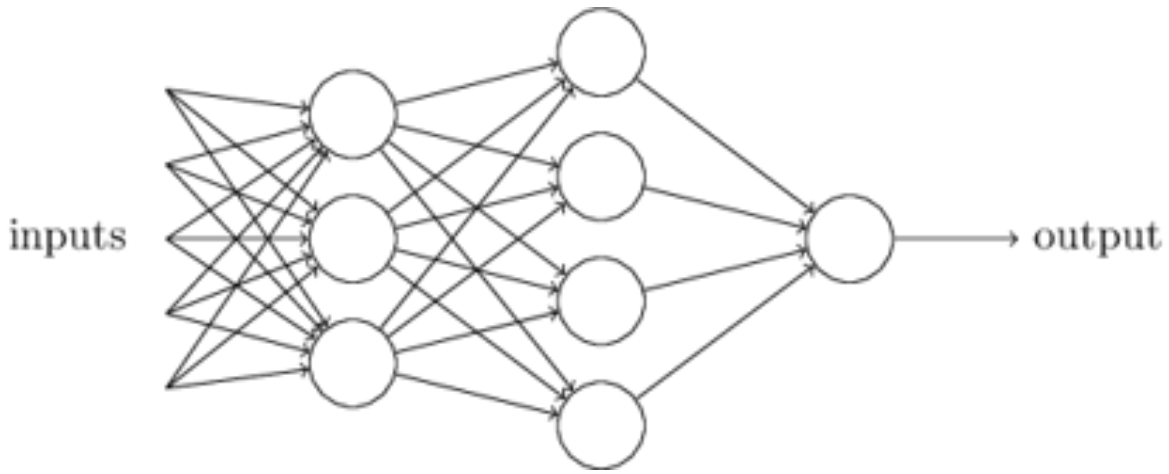
We can graphically represent this decision algorithm as an object that takes 3 binary inputs and produces a single binary output:



This object is called a perceptron when using the type of weighting scheme we just developed.

A network of perceptrons

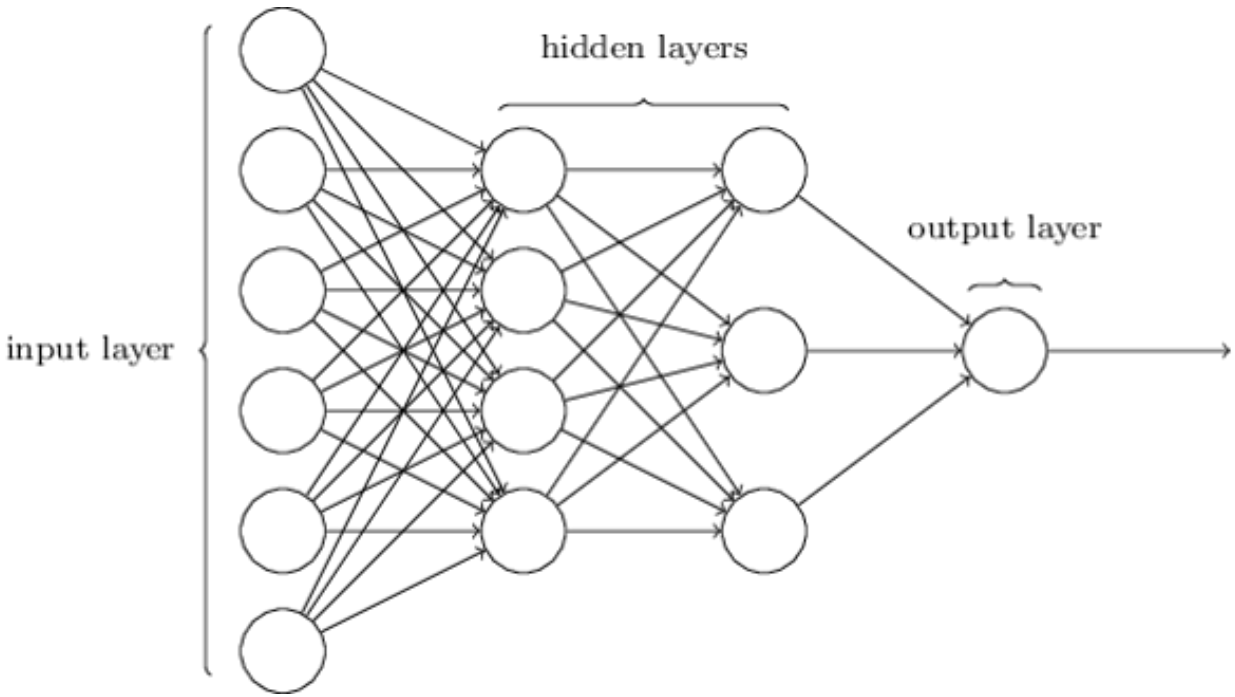
A perceptron takes a number of binary inputs and emits a binary output. Therefore it is easy to build a network of such perceptrons, where the output from some perceptrons are used in the inputs of other perceptrons:



Notice that some perceptrons seem to have multiple output arrows, even though we have defined them as having only one output. This is only meant to indicate that a single output is being sent to multiple new perceptrons.

A network of perceptrons

The input and outputs are typically represented as their own neurons, with the other neurons named hidden layers.

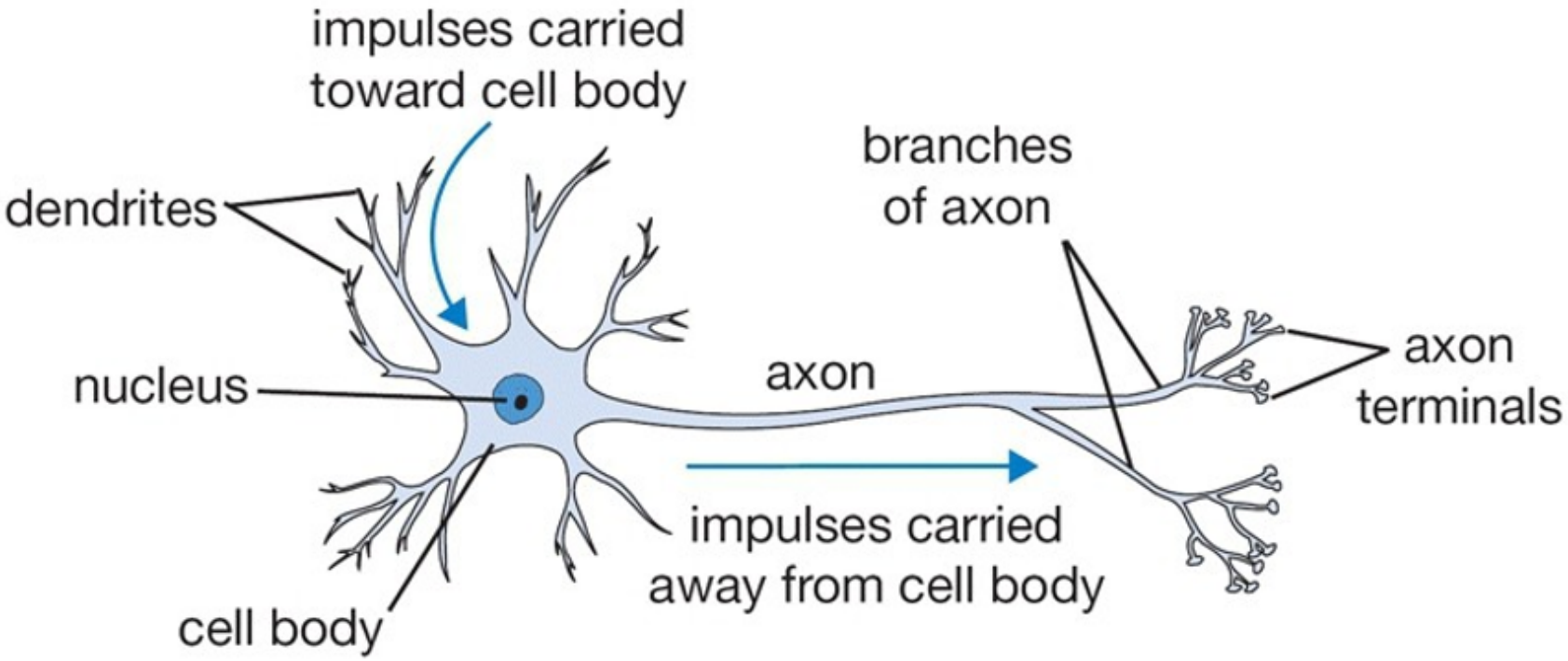


A network of perceptrons, contd.

The biological interpretation of a perceptron is this:

when it emits a 1 this is equivalent to 'firing' an electrical pulse, and when it is 0 this is when it is not firing.

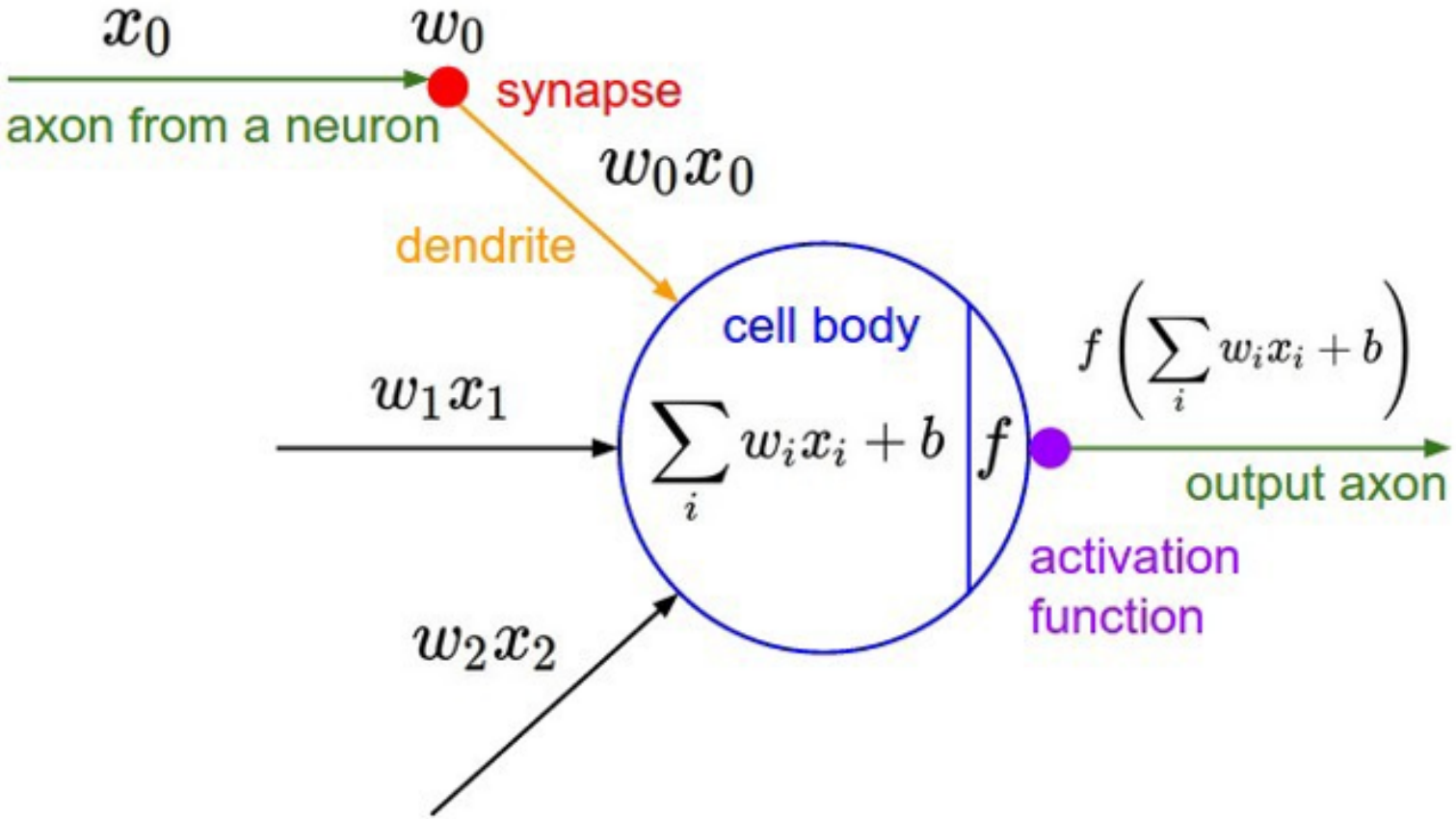
The bias indicates how difficult it is for this particular node to send out a signal.


impulses carried
toward cell body
dendrites
branches
of axon
nucleus
axon
axon
terminals
impulses carried
away from cell body
cell body

Common patterns [1]

- The firing of neurons produces an **activation** that flows through a network of neurons.
- Each neuron receives input from **multiple other neurons** and contributes output in turn to multiple other neurons.
- Local structures in the network can embody **representations** of particular semantic information, though information can be (and is) distributed throughout the network.
- The network can **learn** by changing the connections (and the strength of connections) among neurons.

[1] from


x_0
w_0
synapse
axon from a neuron
$w_0 x_0$
dendrite
cell body
$f\left(\sum_i w_i x_i + b\right)$
$w_1 x_1$
$\sum_i w_i x_i + b$
f
output axon
activation function
$w_2 x_2$

Remember

An Artificial Neural Network is a computational model comprised of interconnected nodes that are inspired by biological neurons.

Keep in mind that this ***neuron*** language is only a metaphor.

A network of perceptrons, contd.

Notice that the network of nodes I have shown only sends signals in one direction.

This is called a feed-forward network.

These are by far the most well-studied types of networks, though there are recurrent neural networks (RNNs) that allow for loops in the network.

The one-directional nature of feed-forward networks is probably the biggest difference between artificial neural networks and their biological equivalent.

Sigmoid neuron

An important shortcoming of a perceptron is that a small change in the input values can cause a large change the output because each node (or neuron) only has two possible states: 0 or 1. A better solution would be to output a continuum of values, e.g. any number between 0 and 1.

Sigmoid neuron, Contd.

As one option, we could simply have the neuron emit the value:

$$\sigma(x \cdot \omega + b) = \frac{1}{1 + e^{-(x \cdot \omega + b)}}$$

For a particularly large positive or negative value of $x\omega + b$, the result will be nearly the same as with the perceptron (i.e., near 0 or 1).

For values close to the boundary of the separating hyperplane, values near 0.5 will be emitted.

Sigmoid neuron, Contd.

This perfectly mimics logistic regression, and in fact uses the logit function to do so. In the neural network literature, the logit function is called the sigmoid function, thus leading to the name sigmoid neuron.

Notice that the previous restriction to binary inputs was not at all needed, and can be easily replaced with continuous input without any changes needed to the formulas.

Activation functions

In the sigmoid neuron example, the choice of what function to use to go from $x \cdot \omega + b$ to an output is called the activation function. Using a logistic, or sigmoid, activation function has some benefits in being able to easily take derivatives and the interpret them using logistic regression.

Other choices have certain benefits that have recently grown in popularity. Some of these include:

1. hyperbolic tan: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = \frac{e^{2z}-1}{e^{2z}+1}$.
2. rectified linear unit: $\mathrm{ReLU}(z) = max(0, z)$
3. leaky rectified linear unit
4. max-out.

Different kinds of nodes can be used and combined in any network.

MLFNN

These units can be comboned in many ways, one of the most basic form is a *multi-layer feed-forward neural network* (MLFFNN).

These networks are designed in layers, each comprised of one or more artificial neurons, or nodes.

- The input layer takes in the data and usually consists of pass-through or purely linear nodes.
- The output layer converts the computation into a form needed in how the network is used.
- In between input and output are one or more hidden layers. A network with more than one hidden layer is often called a deep network.

MLFNN output

For two-class classification problems, for example, a single binary output node does the trick.

Another common example is to produce a probability distribution via the softmax combination. This takes the outputs $z_1, \ldots, z_p$ of several output nodes and transforms the jth output to:

$$z_j \mapsto \frac{e^{z_j}}{\sum_{j=1}^{k} e^{z_j}}$$

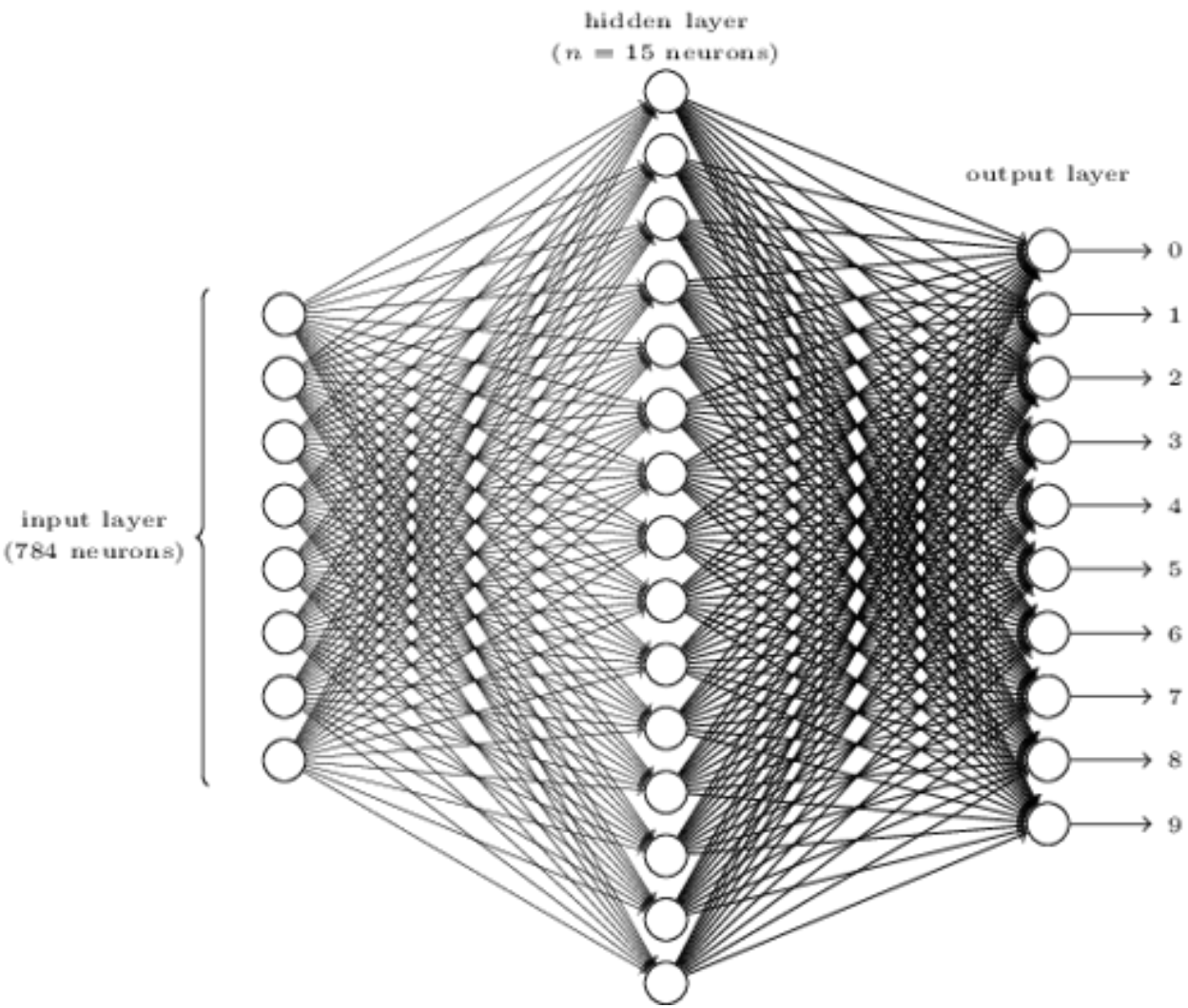
MNIST database

Figure: De-facto 'hello world' data-set for ML and Computer Vision

MNIST

- Every image has p = 28 × 28 = 784 pixels, each of which is an eight-bit grayscale value between 0 and 255 representing the relative amount of the written digit in that tiny square.
- These pixels are stored in the input vector X (in, say, column order).
- The output is the class label, represented by a vector $Y = (Y_0, Y_1, \ldots, Y_9)$ of 10 dummy variables, with a one in the position corresponding to the label, and zeros elsewhere.
- In the machine learning community, this is known as *one-hot encoding*.
- There are 60,000 training images, and 10,000 test images.

MNIST example



To determine which class to put a particular input into, we look at which of the output neurons have the largest output.

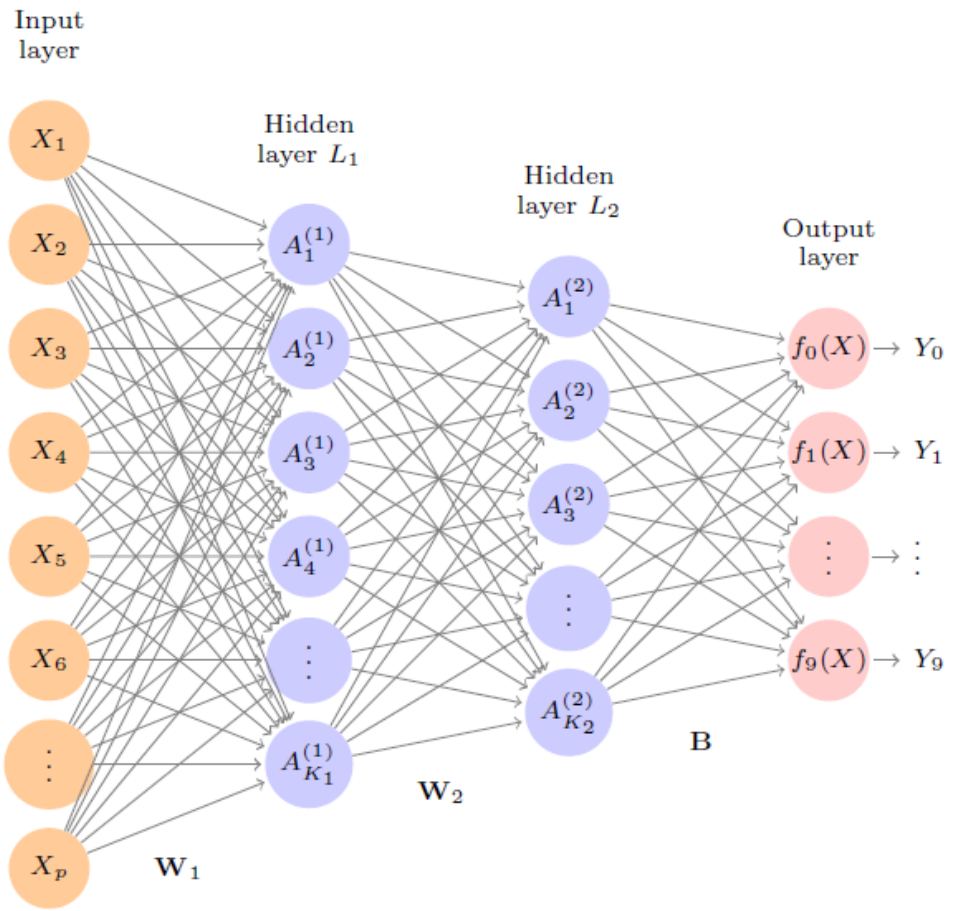


FIGURE 10.4. *Neural network diagram with two hidden layers and multiple outputs, suitable for the* MNIST *handwritten-digit problem. The input layer has* $p = 784$ *units, the two hidden layers* $K_1 = 256$ *and* $K_2 = 128$ *units respectively, and the output layer* 10 *units. Along with intercepts (referred to as* biases *in the deep-learning community) this network has 235,146 parameters (referred to as weights).*

Why so many coefficients?

- Recall each image has 784 pixels. $60,000$ training image, $10,000$ test.
- Two hidden layers L1 (256 units) and L2 (128 units).
- W_1 in Fig **??** represents the entire matrix of weights that feed from the input layer to the first hidden layer L1. This matrix will have $785 \times 256 = 200,960$ elements.
- Each element in $A_k^{(1)}$ feeds to the second hidden layer L2 via the matrix of weights W_2 of dimension $257 \times 128 = 32,896$.
- We now get to the output layer, where we now have ten responses rather than one. The matrix B stores all 129 × 10 = 1,290 of these weights.
- $|W_1| + |W_2| + |B| = 235,146 \gg 60,000($ size of training sample$)$.

Regularization

While this might seem like a large training set, there are almost four times as many coefficients in the neural network model as there are observations in the training set!

To avoid overfitting, some regularization is needed.

In this example, we used two forms of regularization: ridge regularization, which is similar to ridge regression we've studied before, and dropout regularization.

Dropout This is a model selection technique designed to avoid over-fitting in deep learning. This is done by removing input dimensions in X randomly with a given probability p. For example, suppose that we wish to minimize MSE, $\|Y - \hat{Y}\|_2^2$, then, when marginalizing over the randomness, we have a new objective

$$\arg\min_W \mathbb{E}_{D \sim \mathrm{Ber}(p)} \|Y - W(D \times X)\|_2^2 ,$$

Method	Test Error
Neural Network + Ridge Regularization	2.3%
Neural Network + Dropout Regularization	1.8%
Multinomial Logistic Regression	7.2%
Linear Discriminant Analysis	12.7%

TABLE 10.1. *Test error rate on the* `MNIST` *data, for neural networks with two forms of regularization, as well as multinomial logistic regression and linear discriminant analysis. In this example, the extra complexity of the neural network leads to a marked improvement in test error.*

Historical note

- On a historical note, digit recognition problems were the catalyst that accelerated the development of neural network technology in the late 1980s at AT&T Bell Laboratories and elsewhere.
- Pattern recognition tasks of this kind are relatively simple for humans.
- Our visual system occupies a large fraction of our brains, and good recognition is an evolutionary force for survival.
- These tasks are not so simple for machines, and it has taken more than 30 years to refine the neural-network architectures to match human performance.

Learning Neural Networks

We now have an architecture that describes a neural network, but how do we learn the weights and bias terms in the model given a set of training data?

As an important side note, notice that with just one node, we could define a learning algorithm which perfectly replicates a support vector machine or logistic regression.

Cost function

The primary set-up for learning neural networks is to define a cost function (also known as a loss function) that measures how well the network predicts outputs on the test set.

The goal is to then find a set of weights and biases that minimizes the cost.

Cost function, Contd.

One example of a cost function is just squared error loss:

$$C(\omega, b) = \frac{1}{2n} \sum_i (y_i - y(x_i))^2$$

Or, for classification, the hinge loss:

$$C(\omega, b) = \sum_i [1 - y_i - y(x_i)]_+$$

As with the activation functions, there are different cost functions.

Optimization problem

How does one actually do the optimization required in fitting neural networks?

With very few exceptions, every technique is somehow related to gradient descent. That is, we calculate the gradient function, move a small amount in the opposite direction of the gradient (because we are minimizing), and then recalculate the gradient on the new spot.

Gradient descent

Mathematically, we can describe these updates as:

$$\omega_{k+1} = \omega_k - \eta \nabla_\omega C \tag{1}$$
$$b_{k+1} = b_k - \eta \nabla_b C \tag{2}$$

For some value $\eta > 0$.

This tuning parameter is called the learning rate.

Too low, and learning takes a very long time. Too small, and it is likely to have trouble finding the true minimum (as it will keep 'overshooting' it).

Decomposable cost function

One particularly important aspect of all of the cost functions used in neural networks is that it the are able to be decomposed over the samples. That is:

$$C = \frac{1}{n} \sum_i C_i$$

For the individual costs C_i of the i'th sample.

Decomposable cost function, Contd.

Consider now taking a subset $M \subset \{1, 2, \ldots, n\}$ with size m of the training set. It would seem that we can approximate the cost function using only this sub-sample of the data:

$$\frac{\sum_{i \in M} \nabla C_i}{m} \approx \frac{\sum_i \nabla C_i}{n} = \nabla C$$

So it seems that we can perhaps estimate the gradient using only a small subset of the entire training set.

Stochastic gradient descent (SGD)

Stochastic gradient descent uses this idea to speed up the process of doing gradient descent.

Specifically, the input data are randomly partitioned into disjoint groups $M_1, M_2, ..., M_{n/m}$.

We then do the following updates to the weights (biases are done at the same time, but omitted for sake of space):

$$\omega_{k+1} = \omega_k - \frac{\eta}{m} \sum_{i \in M_1} \nabla C_i$$

$$\omega_{k+2} = \omega_{k+1} - \frac{\eta}{m} \sum_{i \in M_2} \nabla C_i$$

$$\vdots$$

$$\omega_{k+n/m+1} = \omega_{k+n/m} - \frac{\eta}{m} \sum_{i \in M_{n/m}} \nabla C_i.$$

Each set M_j is called a **mini-batch** and going through the entire dataset as above is called an *epoch*.

Stochastic gradient descent (SGD), Contd.

The main tuning parameters for this technique are the size of the mini-batch (m), the learning rate (η) and the number of epochs (E) to use.

Note: Some texts refer to SGD as only describing the case with a mini-batch size of 1, whereas others use SGD to refer to this more generic algorithm.

The technical term for the generic algorithm is *mini-batch gradient descent* (MGD), but you will rarely hear the term.

Calculating the gradient function

The last bit that we need in order to run SGD on a neural network is a way to actually calculate the gradient for a given mini-batch.

We will need to derive the equations that allow us to calculate the gradient in a particularly efficient way.

SGD Example: OLS

To simulate stochastic gradient descent, consider using it to find the ordinary least squares estimator of a multivariate regression function:

$$
\begin{aligned}
f(\beta) &= \frac{1}{n} \cdot \sum_i f_i(\beta) \\
&= \frac{1}{n} \cdot \sum_i (y_i - x_i\beta)^2 \\
&= \frac{1}{n} \cdot \sum_i (y_i^2 + \beta^t x_i^t x_i \beta - 2y_i x_i \beta)
\end{aligned}
$$

SGD Example: OLS, cont.

Now, the gradient of f is given by:

$$\begin{aligned}\nabla f &= \frac{1}{n} \cdot \nabla \sum_i f_i(\beta) \\ &= \frac{1}{n} \cdot \sum_i \nabla f_i(\beta) \\ &= \frac{2}{n} \cdot \sum_i (x_i^t x_i \beta - x_i^t y_i)\end{aligned}$$

And can be approximated by:

$$\nabla f \approx \frac{2}{m} \cdot \sum_{i \in M} (x_i^t x_i \beta - x_i^t y_i)$$

For a mini-batch M of size m.

Now we'll implement this in R.

Back-propagation

Based on http://neuralnetworksanddeeplearning.com/chap2.html

Why?

You can treat backpropagation as a black box whose details you're willing to ignore. Why take the time to study those details?

The reason, of course, is understanding.

At the heart of backpropagation is an expression for the partial derivative $\partial C/\partial \omega$ of the cost function C with respect to any weight ω (or bias b) in the network.

The expression tells us how quickly the cost changes when we change the weights and biases. And while the expression is somewhat complex, it also has a beauty to it, with each element having a natural, intuitive interpretation.

And so backpropagation isn't just a fast algorithm for learning. It actually gives us detailed insights into how changing the weights and biases changes the overall behaviour of the network. That's well worth studying in detail.

Neural network review

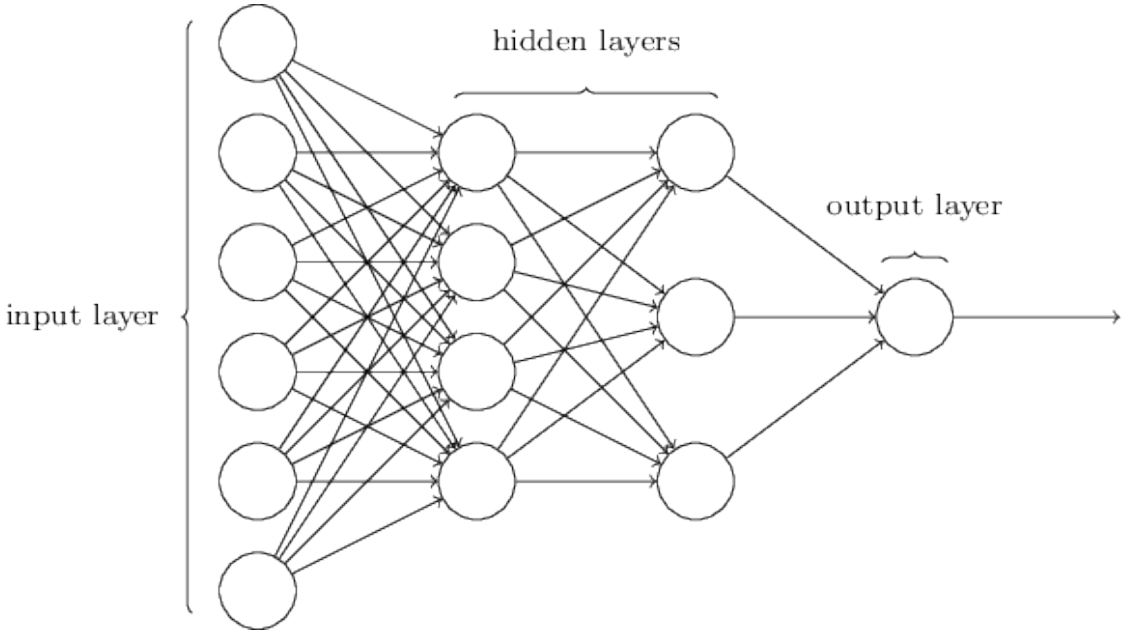
Last time we established the idea of a sigmoid neuron, which takes a vector of numeric variables x and emits a value as follows:

$$\sigma(x \cdot w + b) = \frac{1}{1 + e^{-(x \cdot w + b)}}$$

It is entirely defined by a vector of weights w and bias term b, and functions exactly like logistic regression.

Neural network review, cont.

These single neurons can be strung together to construct a neural network. The input variables are written as special neurons on the left-hand side of the diagram:



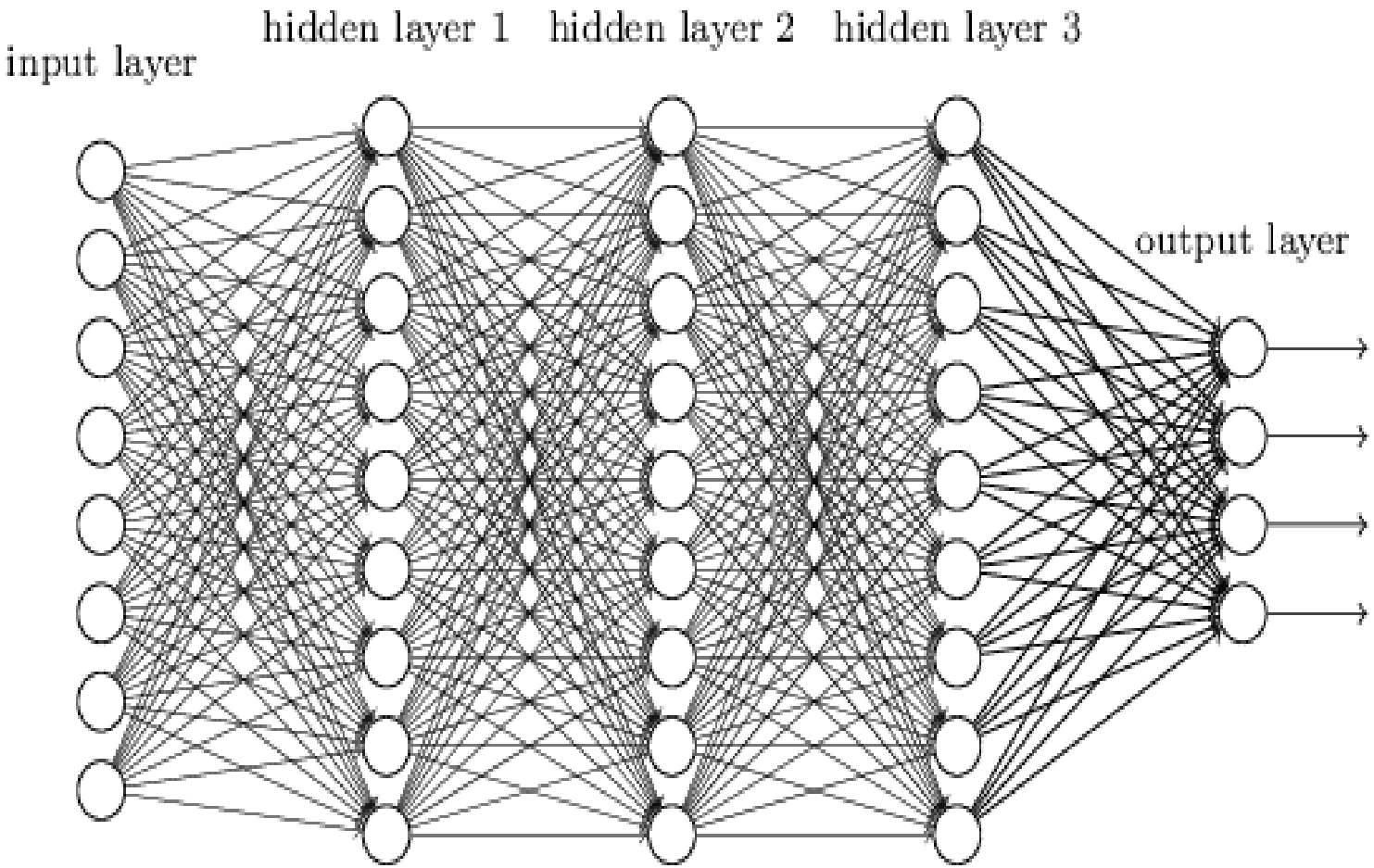
Stochastic gradient descent

We started talking about how to learn neural networks via a variant of gradient descent, called stochastic gradient descent. The only detail left to figure out is exactly how calculate the gradient of the cost function in an efficient way.

Idea behind back-propagation

Before starting down the path of explaining the math behind back-propagation, I want to explain what the big idea behind the method is and why it makes sense as a general approach.

Idea behind back-propagation, cont.

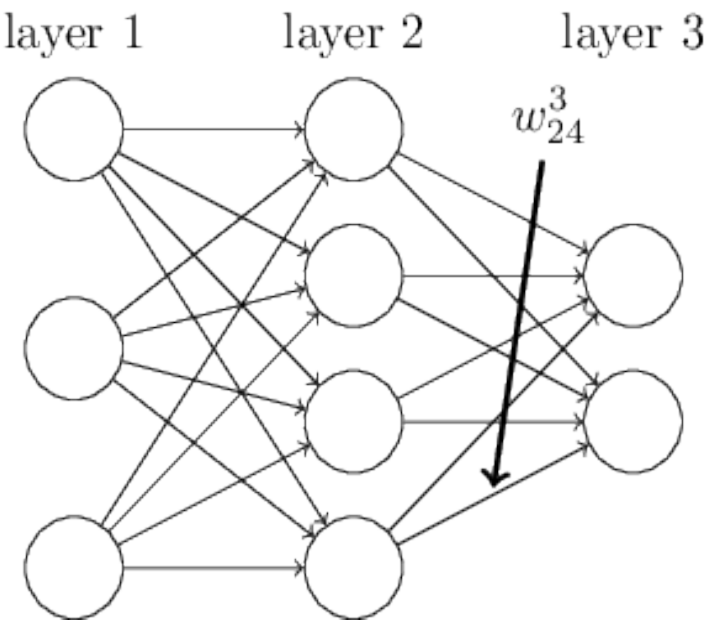


Some notation, cont.

We need a way of referring unambiguously to the weights and biases in a multi-layer neural network. To start define

$$w^l_{j,k}$$

As the weight of node k in layer $l-1$ as applied by node j in layer l.



w^l_{jk} is the weight from the kth neuron in the $(l-1)^{\text{th}}$ layer to the jth neuron in the lth layer

Some notation, cont.

Now let:

$$b_j^l$$

Be the bias of node j of layer l.

Some notation, cont.

We now define two additional quantities. The weighted input emitted from node j in layer l:

$$z_j^l$$

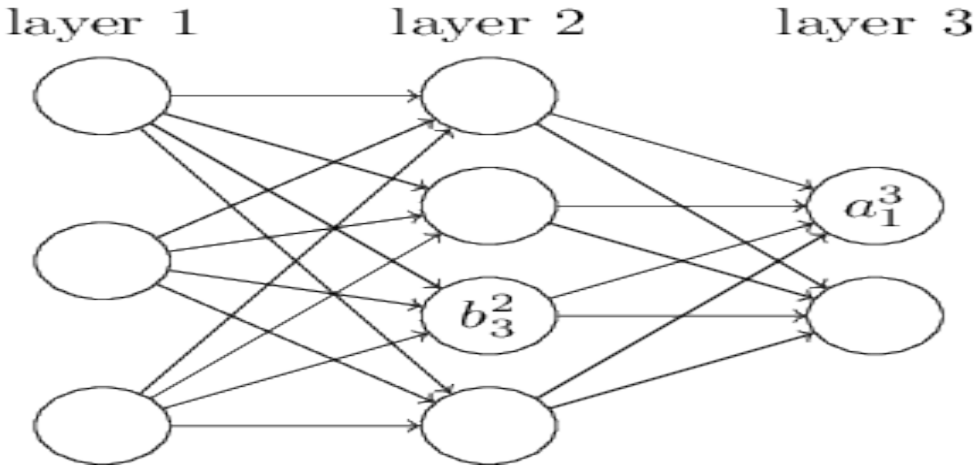
The term input refers to the fact that this in the input to the activation function.

Some notation, cont.

And the activation

$$a_j^l$$

Derived from applying the function σ (the sigmoid or logit function for us so far) to the the weighted input z_j^l.


layer 1
layer 2
layer 3
a_1^3
b_3^2

Some notation, cont.

That's a lot to take in all at once. Here is the cheat-sheet version that shows how all of these quantities fit together:

$$\begin{aligned} a_j^l &= \sigma(z_j^l) \\ &= \sigma(\sum_k w_{jk}^l a_k^{l-1} + b_j^l) \end{aligned}$$

Some notation, cont.

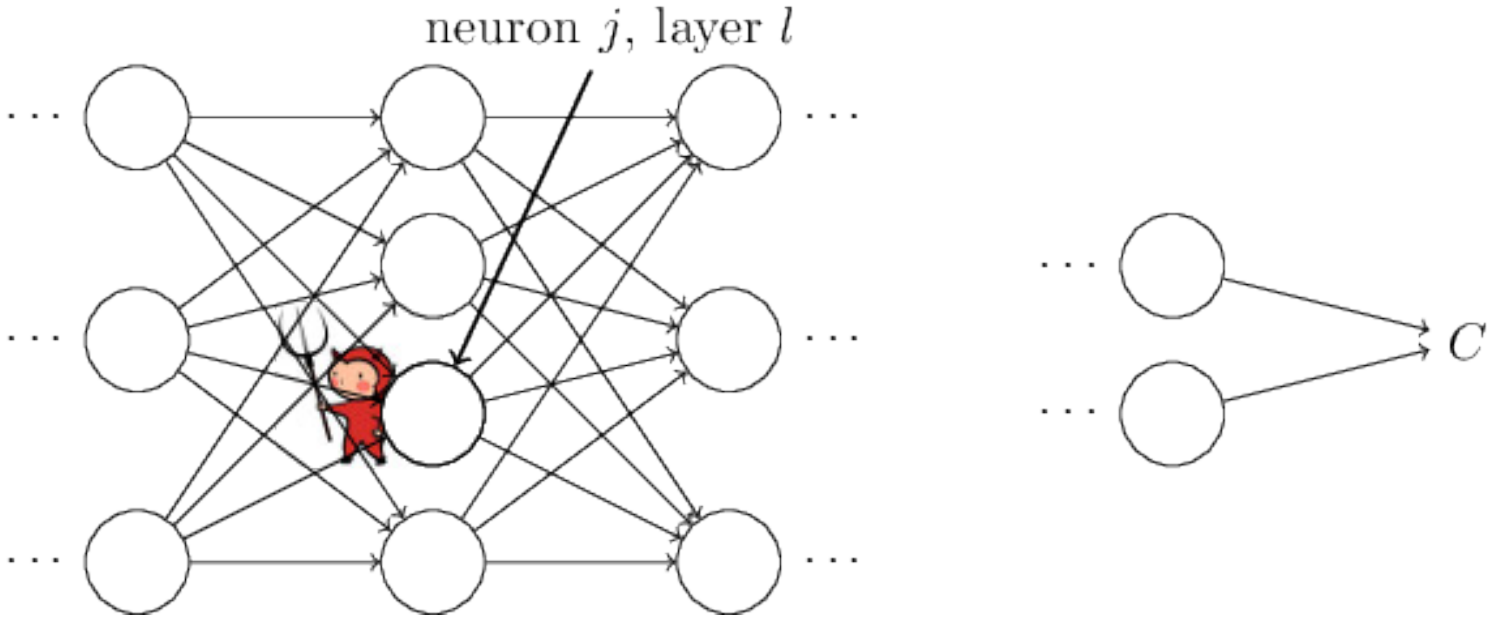
That's a lot to take in all at once. Here is the cheat-sheet version that shows how all of these quantities fit together:

$$\begin{aligned} a_j^l &= \sigma(z_j^l) \\ &= \sigma(\sum_k w_{jk}^l a_k^{l-1} + b_j^l) \end{aligned}$$

It will, also, be beneficial to define one more quantity:

$$\delta_j^l = \frac{\partial C}{\partial z_j^l}$$

Called the error functions. These error functions will help in keeping the number of computations we need to make as small as possible.



The demon sits at the jth neuron in layer l. As the input to the neuron comes in, the demon messes with the neuron's operation. It adds a little change Δz_{lj} to the neuron's weighted input, so that instead of outputting $\sigma(z_{lj})$, the neuron instead outputs $\sigma(z_{lj} + \Delta z_{lj})$.

Cost function again

In what follows we assume that we are dealing with one individual data point. So the cost is simply the cost of that one training point (which we called C_i in Monday's notes).

If the neural network has L total layers, notice that in general we will have the cost being a function only of the activations from layer L of the neural network:

$$C(w, b) = f(a_1^L, a_2^L, \ldots)$$

In our case, this can be explicitly written as:

$$C(w, b) = \sum_k (y_k - a_k^L)^2$$

Note: this sum is over the dimension of the output, not the number of samples! If we only have a univariate output, this will just be a single value.

Feed-forward step

We want to calculate how much the cost changes with respect to the errors on the outer layer. This ends up being a fairly straightforward application of the chain rule:

$$
\begin{aligned}
\delta_j^L &= \frac{\partial C}{\partial z_j^L} \\
&= \sum_k \frac{\partial C}{\partial a_k^L} \cdot \frac{\partial a_k^L}{\partial z_j^L} \\
&= \frac{\partial C}{\partial a_j^L} \cdot \frac{\partial a_j^L}{\partial z_j^L} \\
&= \frac{\partial C}{\partial a_j^L} \cdot \sigma'(z_j^L)
\end{aligned}
$$

Feed-forward step, cont.

We can explicitly calculate the first partial derivative given the cost function. Here for example it is given as:

$$\begin{aligned}\frac{\partial C}{\partial a_j^L} &= \frac{\partial}{\partial a_j^L} \sum_k (y_k - a_k^L)^2 \\ &= 2 \cdot (a_j^L - y_j)\end{aligned}$$

The function σ' is also easily determined by differentiation of the sigmoid function:

$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$$

Back-propagation step

We now want to relate the errors δ_j^l to the errors in δ_j^{l+1}. This allows us to then work backwards from the feed-forward step. Notice that:

$$\begin{aligned}
\delta_j^l &= \frac{\partial C}{\partial z_j^l} \\
&= \sum_k \frac{\partial C}{\partial z_k^{l+1}} \cdot \frac{\partial z_k^{l+1}}{\partial z_j^l} \\
&= \sum_k \delta_k^{l+1} \cdot \frac{\partial z_k^{l+1}}{\partial z_j^l}
\end{aligned}$$

Back-propagation step

We now want to relate the errors δ_j^l to the errors in δ_j^{l+1}. This allows us to then work backwards from the feed-forward step. Notice that:

$$\begin{aligned}
\delta_j^l &= \frac{\partial C}{\partial z_j^l} \\
&= \sum_k \frac{\partial C}{\partial z_k^{l+1}} \cdot \frac{\partial z_k^{l+1}}{\partial z_j^l} \\
&= \sum_k \delta_k^{l+1} \cdot \frac{\partial z_k^{l+1}}{\partial z_j^l}
\end{aligned}$$

To calculate the remaining derivatives, notice that we have a formula relating one set of weighted inputs to the next set:

$$\begin{aligned}
z_k^{l+1} &= \sum_i w_{ki}^{l+1} a_i^l + b_k^{l+1} \\
&= \sum_i w_{ki}^{l+1} \sigma(z_i^l) + b_k^{l+1}
\end{aligned}$$

Back-propagation step, cont.

Using this relationship, we can now take partial derivatives

$$\begin{aligned}\frac{\partial z_k^{l+1}}{\partial z_j^l} &= \frac{\partial}{\partial z_j^l}\left(\sum_i w_{ki}^{l+1}\sigma(z_j^l) + b_k^{l+1}\right) \\ &= w_{kj}^{l+1} \cdot \sigma'(z_j^l)\end{aligned}$$

Which when plugged into our original equations yields:

$$\delta_j^l = \sum_k \delta_k^{l+1} \cdot w_{kj}^{l+1} \cdot \sigma'(z_j^l)$$

Relating errors to weights and bias

We now just need to relate the errors δ_j^l to the gradient with respect to the weights and biases directly. We have, from the linearity of the bias term, a very simple relationship between δ_j^l with the gradient of the bias terms:

$$\begin{aligned}
\delta_j^l &= \frac{\partial C}{\partial z_j^l} \\
&= \frac{\partial C}{\partial b_j^l} \cdot \frac{\partial b_j^l}{\partial z_j^l} \\
&= \frac{\partial C}{\partial b_j^l}
\end{aligned}$$

Relating errors to weights and bias

For the weights, a very similar

$$\begin{aligned}\delta_j^l &= \frac{\partial C}{\partial z_j^l} \\ &= \frac{\partial C}{\partial w_{jk}^l} \cdot \frac{\partial w_{jk}^l}{\partial z_j^l} \\ &= \frac{\partial C}{\partial w_{jk}^l} \cdot \frac{1}{a_k^{l-1}}\end{aligned}$$

Which can be re-written as:

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l$$

Putting these all together

We have now four equations that describe the mechanics of back-propagation. I will write them here a bit more compactly for reference:

$$
\begin{aligned}
\delta_j^L &= \nabla_{a^L} C \circ \sigma'(z^L) \\
\delta_j^l &= ((w^{l+1})^T \delta^{l+1}) \circ \sigma'(z^l) \\
\frac{\partial C}{\partial b_j^l} &= \delta_j^l \\
\frac{\partial C}{\partial w_{jk}^l} &= a_k^{l-1} \delta_j^l
\end{aligned}
$$

Where dropping an index indicates that I am doing math on the entire vector or matrix of values. The symbol $\circ$ (Hadamard product) refers to element-wise multiplication; a common operation in programming but less-so in abstract mathematics.

The back-propagation algorithm

The back-propagation algorithm as a whole is then just:

1. Select an element i from the current minibatch and calculate the weighted inputs z and activations a for every layer using a forward pass through the network
2. Now, use these values to calculate the errors δ for each layer, starting at the last hidden layer and working backwards, using back-propagation
3. Calculate $\nabla_w C_i$ and $\nabla_b C_i$ using the last two equations
4. Repeat for all samples in the minibatch, to get:

$$\frac{\sum_{i \in M} \nabla_w C_i}{|M|} \approx \nabla_w C \qquad \frac{\sum_{i \in M} \nabla_b C_i}{|M|} \approx \nabla_b C$$

5. Update the weights and biases by (the estimates of) $-\eta \cdot \nabla_w C$ and $-\eta \cdot \nabla_b C$, respectively
6. Repeat over every mini-batch, and for the desired number of epochs